

SchemaForge v2.0 — Summary

Autonomous Database Migration Reliability Agent

TrueFoundry × Polaris “Agents That Act” · 26 September 2026

What it is

SchemaForge turns a plain-English migration request into an evidence-backed decision. TrueForge drives the loop: the agent inspects the live PostgreSQL target, maps dependencies, synthesises forward and rollback SQL, rehearses it against a copy of real data, and produces a decision packet. Then it stops. Production is only changed after a human signs an approval bound to that exact migration.

The boundary

Write credentials never sit in the agent loop. Two MCP servers run with different secrets: **sf-core** holds read-only and shadow access; **sf-executor** holds the single production write role and the approval key. A separate terminal tool signs approvals. The executor re-checks the signature, the SQL hash, the schema fingerprint, and a single-use ledger before it applies anything. Forged, edited, expired, replayed, or stale approvals are each rejected with their own code.

Plan

Phase	Time	Outcome
0 Unblock	1.5 h	Config, transactions and demo data actually work
1 Safety core	2.0 h	SQL classifier, signed approvals, executor split
2 Rehearsal	1.5 h	Disposable shadow copies, deterministic verdicts
3 Agent	1.0 h	TrueForge gate, prompt, approval CLI
4 Tests	1.0 h	Twelve tests, one per safety claim
5 Demo	0.5 h	Two dry runs, reset script, backup video
Total	7.5 h	

Demo

Two requests, five minutes. Making `users.email NOT NULL` finds fourteen null rows in rehearsal and returns **DO NOT APPLY** with production untouched. Adding a nullable `archived_at` column passes every check, freezes at the gate, and applies only after the human types the migration hash. Post-apply verification confirms the result. The refusal matters as much as the success.

Done when

- Both scenarios run live against real PostgreSQL, no hand-written SQL.
- The refusal is produced by rule code, not by model opinion.
- Every rejection path has a passing automated test.
- A stranger can clone the repository and reproduce the run.

Where it stops

Six ways a production apply is refused. Each has its own rejection code and its own test.

Attempt	Code	Caught by
Token signed with the wrong key	BAD_SIGNATURE	HMAC verify in the executor
SQL edited after approval	HASH_MISMATCH	Recomputed hash of the SQL
Approval reused	REPLAYED	Single-use ledger in the database
Approval too old	EXPIRED	Short time-to-live on the payload
Schema changed since rehearsal	SCHEMA_DRIFT	Fingerprint recaptured before apply
Prohibited statement	PROHIBITED	Parser, plus a restricted write role

How the verdict is decided

Fixed rules, first match wins. The model explains the verdict; it cannot change it.

Evidence	Verdict
Prohibited or unparseable statement	DO NOT APPLY
Rehearsal failed, or violations found	DO NOT APPLY
Invariant check failed after migration	DO NOT APPLY
Schema drifted, or rollback unproven	REVIEW
Application references flagged	REVIEW
Everything green	APPLY

Stack

TrueForge harness, two MCP servers in TypeScript, PostgreSQL 16 in Docker Compose, a terminal approval tool. No mocked responses: the databases, roles, rehearsal and migration are real.

Not building

Multi-database support, Kubernetes, chat integrations, a login system, a web UI, multi-agent orchestration, or automatic rollback of changes that cannot be reversed.